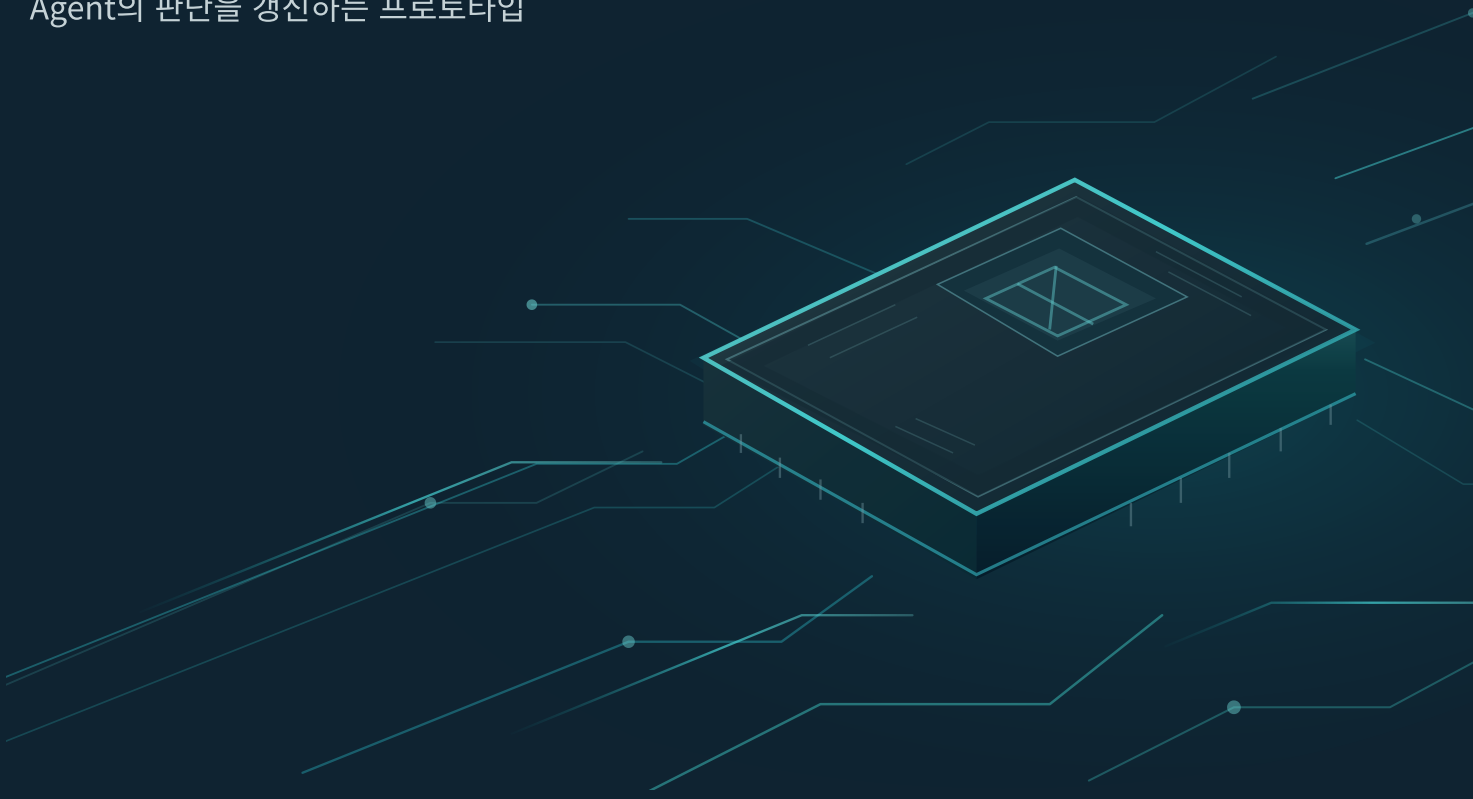
Software Engineer, Compiler (AX Engineer)

2차 직무 과제 전형 보고서

팀 Agent를 위한 공유 지식 저장소 설계와 검증

Hindsight와 Git을 바탕으로 검증된 지식을 공유하고, 새로운 근거에 따라 지식과 Agent의 판단을 갱신하는 프로토타입



APPLICANT
지원자 박승우

RECRUITMENT STAGE
2차 직무 과제 전형

VERSION
제출 보고서

팀 지식 공유 검증 개요

설정 오류 사례를 통해 지식의 제안·검토·게시·활용·갱신을 검증했습니다.

해결하려는 문제

개인의 Agent가 알아낸 해결책은 대화가 끝나면 다른 구성원이 찾기 어렵습니다. 반대로 확인되지 않은 내용을 무작정 공유하면 오래된 정보나 잘못된 조건이 팀에 퍼질 수 있습니다. 이 프로젝트는 **어떤 지식을 공유하고, 누가 확인하며, 나중에 어떻게 고칠지**에 집중합니다.

가상의 팀과 사례

A는 설정 오류가 발생해도 빠진 항목을 알 수 없는 문제를 개선합니다. B의 Agent는 공유된 지식을 검색하고 원본을 확인한 뒤 자신의 예제에 적용합니다. 이후 다른 필수 항목이 추가되자 팀 지식을 갱신하고 다시 확인합니다.

> **검증 요약.** 지식 ID 1건을 v1→v2로 갱신했고, 실제 지식 PR 2건을 사용자가 병합했습니다. 서로 다른 코드·검증 단위의 테스트 총 36개(21+5+5+5)가 통과했습니다. 전체가 각각 독립적인 종단 간 테스트라는 뜻은 아닙니다. 별도 Agent의 v1 활용과 기록을 전달받은 새 세션의 v2 적용을 확인했습니다.

표기: v1/v2는 첫 버전과 갱신 버전을 뜻합니다. 제출본의 지식 파일과 기록은 버전 이름을 통일한 사본이며, GitHub와 Hindsight의 과거 원본 표기는 다를 수 있습니다. 아래 화면은 원본의 일부를 잘라 표시했습니다.

읽는 순서

1. 실행할 코드와 결과 **2.** 설계와 검증 방식 **3.** 실제 사용 및 결과
4. 구현 범위와 한계 **5.** AI Coding Agent 활용 내역

이 저장소는 설정 오류 사례만 옮긴 제출용 스냅샷입니다. 실제 실험의 PR과 커밋은 별도의 공개 원본 저장소에 보존했습니다. 제출용 코드와 실행 방법은 제출용 저장소에 있습니다.

1. 프로토타입

실행과 결과 확인

표준 라이브러리만으로 예제를 실행하고, 저장소 검증은 Python 3.12 가상환경에서 수행합니다.

예제: 설정값이 없을 때

조건	실행 결과
기존: model 누락	설정 오류
v1: model만 누락	설정 오류: model 항목이 없습니다
v2: model·version 누락	설정 오류: model, version 항목이 없습니다
v2: 두 항목 모두 있음	설정 정상

v1 행은 version의 존재 여부를 다루지 않은 당시 결과입니다. v2의 model 누락 예시는 version이 있는 경우입니다.

새 저장소에서 실행

```
git clone https://github.com/5jyan/team-agent-knowledge-showcase.git
cd team-agent-knowledge-showcase
python3.12 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
python scripts/validate.py knowledge
python -m unittest discover -s tests -v
python -m unittest discover -s examples/config-validation -p 'test_*.py' -v
(cd experiments/config-agent-b && python -m unittest discover -s tests -p 'test_*.py' -v)
python -m unittest discover -s experiments/blind-discovery -p 'test_*.py' -v
python examples/config-validation/run_demo.py
```

새로 복제한 환경에서 지식 1건, 핵심 테스트 21개, 설정 예제 테스트 5개, B 코드 테스트 5개, 추가 검색 실험 테스트 5개가 통과했습니다. 서로 다른 코드와 검증 단위의 결과이며 36개 전부가 독립적인 종단 간 검증은 아닙니다. `README.md`에 전체 명령을 적었습니다.

> 위 예제 실행은 AI Agent나 Hindsight를 호출하지 않습니다. 원격 근거는 `python scripts/validate.py knowledge --verify-remote`로 별도 확인할 수 있으나 네트워크·GitHub 접근 환경에 따라 실패할 수 있습니다. 이 검증은 커밋 접근 여부만 확인하며 실제 테스트 결과의 정확성까지 보장하지 않습니다.

GitHub PR과 팀 지식 게시 재현

제출 ZIP은 코드 검증용입니다. GitHub PR과 Hindsight 게시는 자신의 Fork를 clone한 Git 저장소에서 재현합니다.

---

1. 자신의 Fork 준비

제출용 공개 저장소에서 **Fork**를 눌러 본인 계정에 복사합니다. 이후 PR의 대상(Base)은 원본 작성자의 저장소가 아니라 **본인 Fork의 main**으로 설정합니다. 기존 PR #5·#6의 이력은 별도 실제 원본 저장소에 보존돼 있습니다.

2. 변경 Branch → PR → 자동 검사

```
GITHUB_USER=YOUR_GITHUB_ID
git clone "https://github.com/$GITHUB_USER/team-agent-knowledge-showcase.git"
cd team-agent-knowledge-showcase
git switch -c demo/review-date
# knowledge/COMP-015.yaml의 maintenance.review_after만 수정
python scripts/validate.py knowledge
git add knowledge/COMP-015.yaml && git commit -m "docs: revise knowledge review date"
git push -u origin demo/review-date
```

GitHub에서 **base: 본인 Fork의 main**, compare: `demo/review-date`로 PR을 생성합니다. Actions의 `validate-knowledge` 상태 검사가 통과하면 diff·근거·적용 범위를 사람이 검토하고 병합합니다. 단순 날짜 변경은 PR/게시 경로 재현용이며 신규 bugfix의 실증은 아닙니다.

3. GitHub 보호 규칙과 실제 승인

Settings → Branches / Rulesets에서 main에 PR 필수, `validate-knowledge` 필수 검사, 강제 Push 차단을 설정합니다. 독립 리뷰어가 있다면 승인 1건을 요구할 수 있습니다. **본인이 만든 PR을 본인이 병합하는 것은 독립된 리뷰어 승인과 다릅니다.** Fork에서 Actions가 꺼져 있으면 활성화해야 합니다.

4. 병합 후 Hindsight 게시 — 현재 구현

```
git switch main && git pull --ff-only
python scripts/reconcile.py --snapshot-commit "$(git rev-parse HEAD)" \
  --source-repository "$GITHUB_USER/team-agent-knowledge-showcase" \
  --bank-id team-knowledge-repro --dry-run
```

로컬 Hindsight 1.1.0에서 새 Bank(`team-knowledge-repro`)를 만든 뒤 `--dry-run`을 제거해 동일 명령을 실행합니다. 실제 동기화와 검색 확인이 필요하며, PR 병합만으로 Hindsight가 갱신되지는 않습니다.

구현 범위. 현재 Actions는 PR 검증만 수행하며 Merge 후 게시 API는 수동입니다. ZIP은 Git 이력이 없어 게시에 사용할 수 없습니다. **상세 재현 절차:** `docs/GITHUB_SETUP.md`. 머지 후 자동화·인증·실행기 설계: `docs/AUTOMATION_DESIGN.md`.

동일 입력의 v1·v2 비교

원본 Git의 실제 두 버전을 같은 입력으로 실행했습니다. v2에서 version이 필수 항목으로 추가된 조건을 보여줍니다.

같은 입력	v1 당시 결과	v2 결과
`{}`	model 누락 안내	model, version 누락 안내
`{"model":"demo"}`	설정 정상	version 누락 안내
`{"model":"demo","version":"1"}`	설정 정상	설정 정상

왜 갱신해야 했나

두 번째 입력에서 v1은 설정 정상을 반환하지만, 새 요구사항인 v2는 version 누락을 안내합니다. B가 이전 팀 지식만 계속 사용한다면 바뀐 요구사항을 놓칠 수 있습니다.

이는 v1이 당시부터 틀렸다는 뜻이 아닙니다. v1은 model만 검사했고, version 필수 조건은 v2에서 추가됐습니다.

재현 근거

원본 저장소의 v1 커밋 a54e8d6…, v2 커밋 f15b484…에서 같은 함수의 실제 코드를 읽어 실행했습니다. 저장소 README의 명령과 아래 스크립트로 비교할 수 있습니다.

> `experiments/revision-comparison/compare.py`

예제 코드를 실제로 실행하는 스크립트이므로 신뢰한 원본 checkout만 대상으로 사용합니다. 이 결과는 가상 설정 예제에 한정됩니다.

요구사항에서 기능을 도출한 이유

Hindsight는 기억의 저장·검색·갱신을 이미 제공한다. 이번 과제에서는 기억 엔진을 다시 만드는 대신, **어떤 지식을 팀에 공개할지 검증하고 승인·갱신하는 과정**을 추가했다.

> **설계의 핵심 결정.** Git `main`의 승인된 YAML을 유일한 지식 원본으로 두고, Hindsight는 Agent가 자연어로 검색하는 파생 저장소로 사용한다. Agent는 제안하고 사람은 근거를 검토한다.

과제에서 필요한 것	기존에 제공되는 기능	추가한 정책·구현과 선택 이유
개인 경험과 팀 지식 구분	Hindsight의 Bank 저장·검색	미검증 메모는 개인 기록으로 두고, 증거를 갖춘 내용만 `share-knowledge` Skill로 후보화. **분류 정책·Skill 구현**; 사용자별 ACL은 미구현.
재사용 가능한 지식의 품질	Git의 변경 이력 및 Hindsight의 기억 저장	적용 범위·제외 조건·테스트 근거·재검토 시점을 YAML로 구조화. `validate.py`와 CI로 형식을 검사하고 의미·근거는 사람이 검토.
팀에 공유하기 전 승인	GitHub Branch·PR·Actions	Agent의 제안은 Draft PR로 제한하고, 병합된 `main`만 게시. **PR/CI 흐름 구현**; 독립 리뷰어 강제는 미구현.
다른 Agent가 찾아서 활용	Hindsight Recall·MCP 검색	검색 결과를 그대로 정답으로 취급하지 않고 Git 원본의 ID·상태·적용 범위를 다시 확인하도록 지침 작성. **Agent B 사례 검증**; 서버 강제 통제 아님.
새 근거에 따른 갱신·폐기	Hindsight의 동일 문서 교체·삭제 API	같은 `COMP-015` ID를 유지한 v1→v2 변경 PR과 `reconcile.py`로 반영. **수동 게시 구현**; 이전 세션 기억 자동 수정은 미검증.
지속 가능한 관리·추적	Git 이력과 Hindsight 문서 메타데이터	지식마다 담당자·검토일·갱신 조건·출처 커밋을 기록. **스키마와 운영 절차 구현**; 자동 재검증 알림은 후속 과제.
팀 운영 자동화·접근 제어	GitHub의 이벤트 실행과 Hindsight REST API	PR 자동 검사까지만 구현. Merge→게시 자동화, 인증된 쓰기 권한·Bank별 ACL·실패 알림은 **확장 설계만 작성**.

따라서 무엇을 직접 만들었나

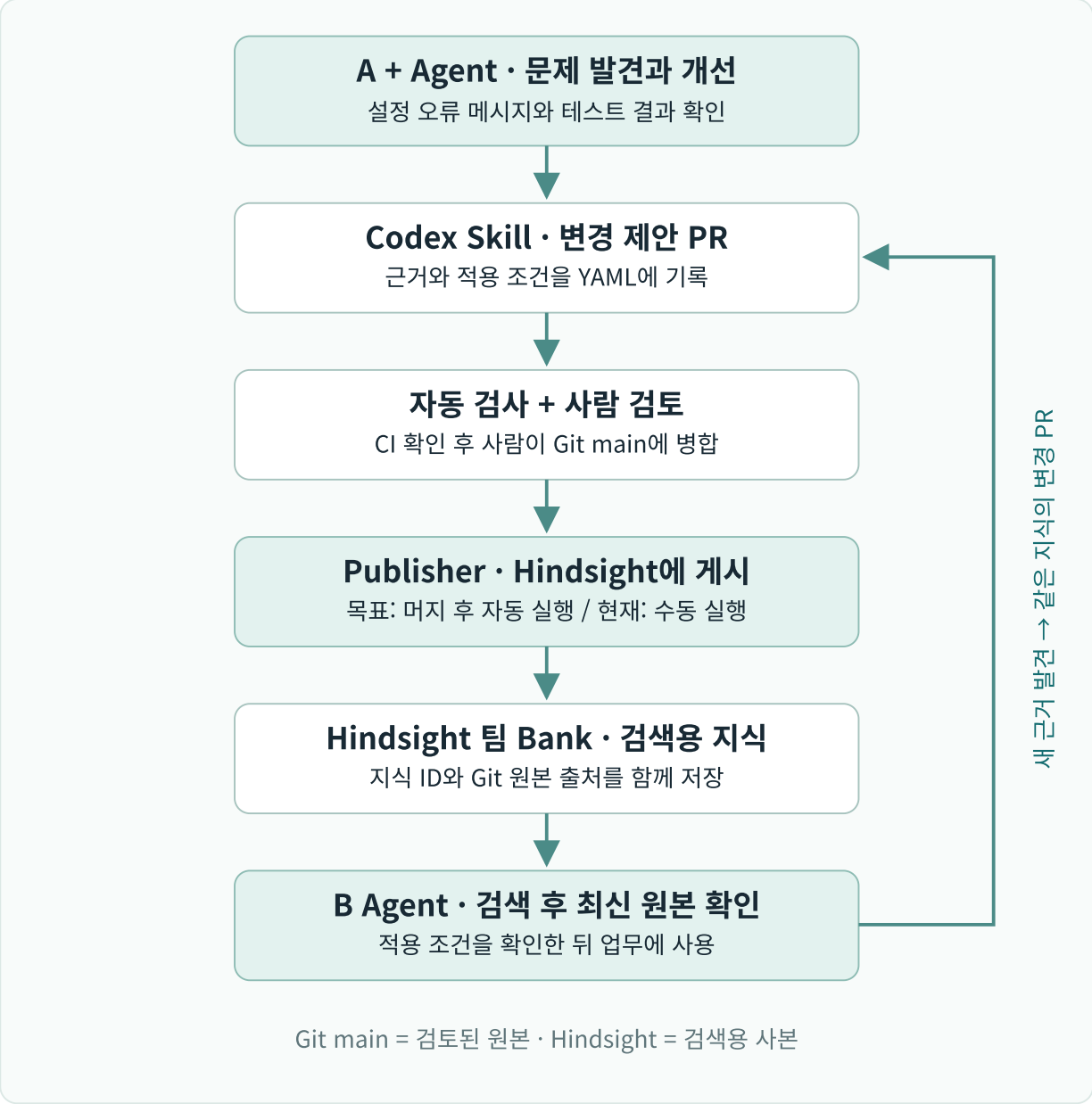
구현: 지식 YAML/Schema · 검증기와 GitHub CI · Codex 공유 제안 Skill · Git 승인 원본을 Hindsight에 반영하는 수동 Publisher · 실제 v1/v2 적용 실험.

재사용: Hindsight의 Bank/Recall/문서 API와 GitHub의 PR/Commit/Actions. **후속 설계:** 자동 게시, 서버 ACL, 독립 승인 강제, 주기적 재검증.

검증 범위: 본 프로토타입은 가상의 설정 검사 사례 1건이다. 개인 기억만 사용하거나 Git 문서만 사용하는 방식에 대한 반복적 성능 우위 실험은 수행하지 않았다. 구현 상태와 실제 근거는 이어지는 설계·검증 페이지에 구분해 명시한다.

팀 지식 공유 흐름

Git은 사람이 검토한 원본이고, Hindsight는 해당 지식을 찾기 위한 검색 공간입니다.



현재 구현은 GitHub 검사 후 사람이 병합하고, 운영자가 Publisher를 직접 실행합니다. 목표인 머지 후 자동 게시 경로는 아직 구현하지 않았습니다.

2. 설계 문서

개인 기록과 팀 지식의 기준

검증되지 않은 개인 메모를 바로 팀의 규칙으로 취급하지 않습니다.

무엇을 어디에 두나

구분	내용과 사용 기준
개인 Bank	작업 중 관찰, 미검증 가설, 개인 메모. 팀의 승인된 규칙이 아닌 탐색 단서입니다.
공유 후보	다른 작업에도 쓸모가 있고, 재현 방법·근거·적용 조건을 설명할 수 있는 내용. PR에서는 아직 제안입니다.
팀 지식	사람이 검토해 Git main에 병합한 YAML. 검색을 위해 Hindsight 팀 Bank에 별도 게시합니다.

지식 파일의 구성

`knowledge/COMP-015.yaml`은 ID·revision·상태, 문제와 해결, 적용 조건과 예외, 실행 명령·결과·근거 커밋, 담당자와 재검토 계기를 포함합니다. `templates/bugfix.yaml`은 작성 예시, `schemas/knowledge.schema.yaml`은 필수 형식의 기준입니다.

스키마에 제안 전용 상태가 없어 PR 후보에도 `active`가 들어갈 수 있습니다. 공식 지식인지 여부는 상태 값 하나가 아니라 **사람이 main에 병합한 원본인지**로 판단합니다.

어떻게 함께 쓰나

B의 Agent는 관련 기억을 검색한 뒤 Git 원본의 ID·상태·버전·범위·근거를 확인합니다. 원본이 없거나 자신의 업무 조건과 맞는지 확인할 수 없다면 적용을 보류하도록 지시합니다. 이는 Agent의 작업 지침이며 서버가 모든 Agent에게 강제하는 통제는 아닙니다.

> Bank 이름 구분은 논리적 분리입니다. 현재 서버 측 사용자별 접근 권한과 Publisher 전용 쓰기 권한은 구현·검증하지 않았습니다. 별도 사후 실험에서 개인 Bank에만 저장한 가상 메모가 개인 문서 목록과 검색에는 있었고 팀 문서 목록·동일 질의에는 없었습니다. 이는 라우팅 확인이지 접근 차단이나 과거 PR의 개인 Bank 기원을 입증하지 않습니다.

2. 설계 문서

Skill, PR, 검토자의 역할

Agent는 근거 있는 제안을 준비하고, 반영 여부는 사람이 결정합니다.

Skill이 수행하는 일

Codex의 `$share-knowledge` 지침은 기존 YAML과 스키마 확인 → 근거 수집 → 후보 작성 → 검사 → draft PR 생성 순서로 구성됩니다. 저장소의 `.agents/skills/share-knowledge/SKILL.md`는 원래 스킬을 제출용 경로에 맞게 수정한 사본입니다. 스킬은 PR을 승인·병합하거나 팀 Bank에 직접 쓰지 않습니다. 설정 예제의 첫 Skill CLI 실행은 중단됐고, 이후 Codex 앱이 절차를 이어 PR #5를 완성했습니다. Skill 단독으로 처음부터 PR까지 완료된 실행은 이 사례에서 확인하지 못했습니다.

PR에 담은 정보

항목	설정 오류 사례
문제와 개선	일반적인 오류 → 빠진 설정 항목을 알려주는 오류.
근거	실행 가능한 예제, 테스트 명령과 결과, 코드 커밋.
범위·제외	가상의 설정 검사 예제로 한정. 값의 유효성이나 실제 제품 전체는 보장하지 않음.
후속 단계	사람의 병합 후 Publisher가 게시. PR 작성만으로 팀 Bank는 바뀌지 않음.

무엇을 누가 확인하나

검증기는 필수 필드, 형식, ID와 일부 비밀 패턴을 검사합니다. GitHub Actions는 Python 3.12에서 검사와 테스트를 실행합니다. 사람은 테스트가 주장과 맞는지, 적용 범위와 민감정보 처리에 문제가 없는지 확인하고 병합합니다.

원격 근거 커밋 확인은 `--verify-remote`를 별도로 실행합니다. 자동 검사 성공은 사실의 의미적 정확성이나 독립 리뷰를 증명하지 않습니다. 이 데모의 PR은 제출자가 직접 병합했습니다.

2. 설계 문서

지식 게시와 갱신 방식

현재는 머지 후 Publisher를 수동 실행합니다.

현재 구현

운영자가 최신 Git main을 받은 뒤 `scripts/reconcile.py`를 실행합니다. Publisher는 로컬·원격 main과 저장소가 일치하는지, 추적 파일이 깨끗한지, 해당 커밋의 YAML이 스키마에 맞는지 확인합니다.

기존 Bank 문서와 비교해 추가·갱신·삭제 대상을 정합니다. Hindsight에는 `POST /v1/default/banks/{bank_id}/memories`로 지식 본문, 문서 ID, Git 원본 경로와 커밋을 보냅니다. 같은 ID의 변경은 교체하고, 다른 저장소가 관리하는 문서는 덮어쓰지 않습니다.

실패와 오래된 기억

모든 요청과 마지막 main 재확인이 성공한 뒤 완료 기록을 남깁니다. 도중에 실패하면 일부 문서는 이미 바뀌었을 수 있어 운영자가 최신 상태를 확인하고 다시 실행해야 합니다. 게시나 삭제가 열린 대화·개인 기억·별도 캐시까지 자동으로 바꾸는 것은 아닙니다.

자동 게시로 확장하려면

목표는 **검토된 PR이 main에 병합 → 게시 작업 실행 → 결과 확인·실패 알림**입니다. 현재 GitHub Actions에는 이 작업이 없습니다. 자동화하려면 로컬 Hindsight에 접근할 수 있는 실행 환경, 서버 인증·게시 전용 권한, 중복 실행과 순서 제어, 재시도와 감사가 필요합니다.

승인과 병합은 다릅니다. 승인만으로는 main이 바뀌지 않습니다. 또한 main 직접 push도 별도로 통제해야 '검토된 PR만 게시'하는 정책을 보장할 수 있습니다.

2. 설계 문서

기본 기능과 추가 설계

오픈소스가 제공하는 기능과 이 프로젝트에서 추가한 책임을 분리합니다.

구분	담당 기능
Hindsight 기본	Bank·문서 저장, 기억 추출과 recall 검색, 문서 갱신·삭제 API.
GitHub 기본	파일·커밋 이력, PR과 CI 실행.
프로젝트 구현	지식 스키마·검증기, Codex Skill, 수동 Publisher, 예제와 테스트.
운영 절차	사람의 근거 검토, Agent의 원본 재확인, 담당자의 재검토.
추가 설계	자동 게시, Bank별 ACL, 독립 리뷰 강제, 분산 잠금과 장애 알림.

대안과 선택 이유

개인 기억만 사용하면 각자의 작업 맥락은 보존하지만 다른 사람이 최신 근거를 확인할 공유 절차가 부족합니다. **Git 문서만 사용**하면 원본과 이력이 명확하고 단순하지만 관련 문서를 찾는 방법이 별도로 필요할 수 있습니다.

Hindsight와 Git을 결합하면 자연어로 후보를 찾고 원본을 확인할 수 있습니다. 대신 게시 지연과 추가 운영이 필요합니다. 이 사례에서 세 방식을 반복 비교하지 않았으므로 정확도·시간·비용 우위는 주장하지 않습니다.

3. 사용 예시 및 확인 결과

첫 지식 제안과 병합

A 역할의 작업에서 model 누락 메시지를 개선해 COMP-015 v1로 제안했습니다.

같은 입력으로 변경 전후 메시지를 비교하고, model 누락·정상 입력·설정값 미노출 테스트 3개를 확인했습니다. 코드와 테스트의 실제 커밋을 지식 파일에 근거로 기록한 뒤, 사용자가 원본 저장소의 PR #5를 병합했습니다.

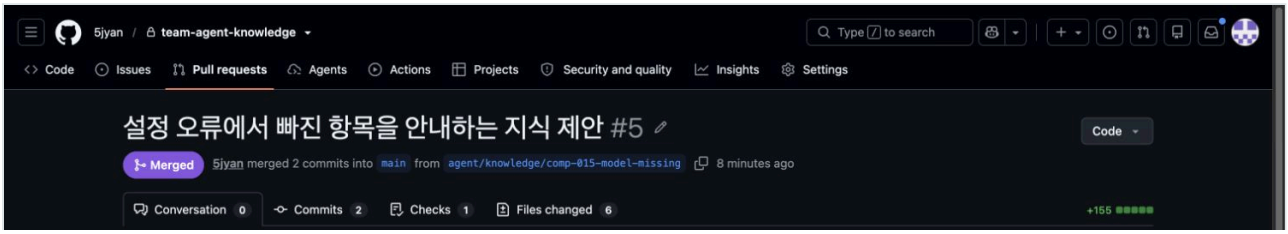


그림 1. PR #5의 실제 Merged 화면 일부. 저장소 공개 전 촬영했으며, 원문은 아래 링크에서 확인할 수 있습니다.

원본: PR #5 · 사용자가 직접 병합했으며 별도 제3자 리뷰는 확인하지 않았습니다.

3. 사용 예시 및 확인 결과

별도 Agent의 지식 활용

별도 Codex 세션이 B 역할로 팀 Bank와 Git 원본을 확인했습니다.

게시와 검색

PR #5 병합 후 Publisher를 수동 실행해 `COMP-015 v1`을 원본 저장소의 `team-compiler` Bank에 게시했습니다. B는 Bank를 검색하고 문서 ID와 출처를 확인한 뒤 Git 원본의 상태와 적용 범위를 읽었습니다. 당시 검색 결과와 Git main의 커밋은 모두 `a54e8d6…`이었습니다.

업무에 적용한 결과

B는 자신의 독립 예제에 model 누락 오류를 작성하고 테스트했습니다. 테스트 4개가 통과했습니다. 이는 비슷한 가상 예제에 대한 적용이며, 다른 실제 서비스의 동작을 보장하지 않습니다.

v1이 답하지 못한 질문

“model과 version이 모두 없으면 두 항목을 표시해도 되는가?”라는 질문에 B는 팀의 승인된 규칙으로 확정하지 않았습니다. v1이 version을 범위 밖으로 명시했기 때문입니다. 당시 B의 함수가 빈 설정에서 model만 표시한 사실과, 팀이 두 항목 표시를 승인했는지는 별개입니다.

> **확인한 경계.** 검색 결과가 있다는 이유만으로 적용 범위 밖의 행동을 승인된 규칙이라 판단하지 않았습니다.

기록: `experiments/config-agent-b/trace.md`. B는 별도 Codex 세션이며 실제 다른 팀원의 계정은 아닙니다.

3. 사용 예시 및 확인 결과

신규 조건 반영과 지식 갱신

version도 필수인 새 조건을 테스트하고 같은 지식 ID를 v2로 수정했습니다.

---

v2 후보는 `model`과 `version`을 모두 검사하고 누락된 항목을 한 번에 안내합니다. 각각 하나 누락, 동시 누락, 정상 입력, 값 미노출을 포함한 테스트 5개가 통과했고 사용자가 PR #6을 병합했습니다.

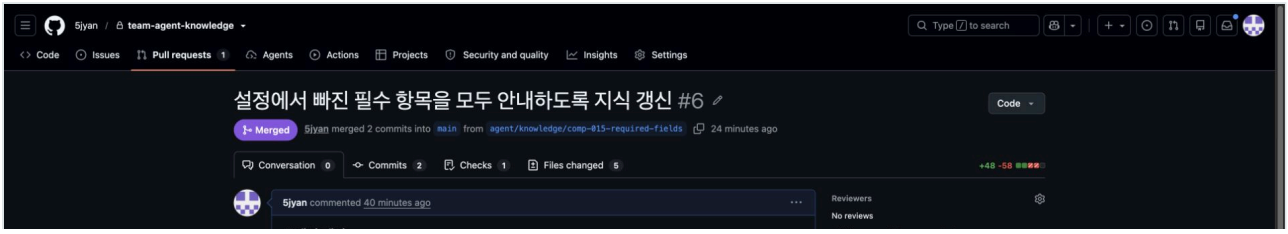


그림 2. 갱신 PR #6의 실제 Merged 화면 일부. 전체 내용과 테스트 근거는 아래 원본 링크에서 확인할 수 있습니다.

원본: PR #6 · 갱신은 예제의 두 키 존재 여부에 한정합니다.

3. 사용 예시 및 확인 결과

갱신 지식 검색 결과

과거 실험 이후 최신 제출용 Bank에 게시한 v2 내용을 Hindsight에서 다시 검색한 화면입니다.

운영자가 수동 게시한 뒤 `COMP-015 model version 설정 오류 누락 항목`으로 검색했습니다. 화면에 4개의 결과가 표시됐고 첫 결과에 v2의 두 항목 누락 규칙이 포함됐습니다.

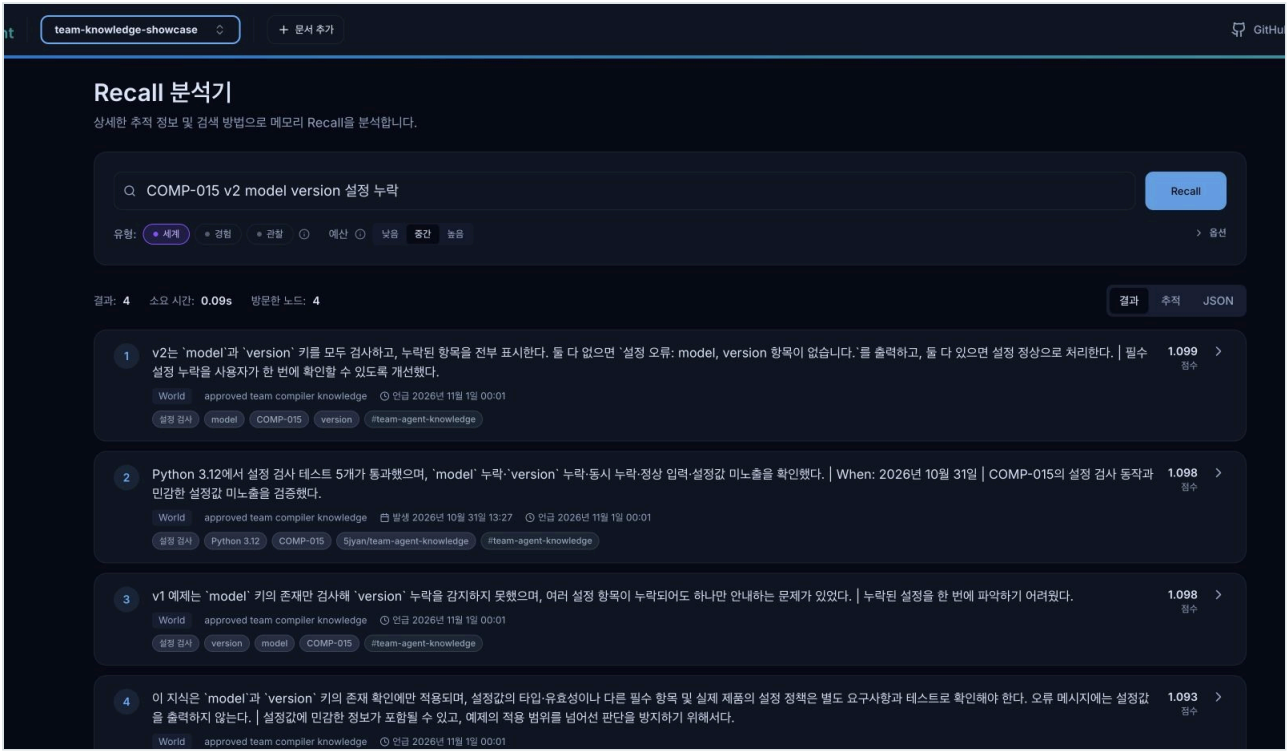


그림 3. 2026-11-01 현재 제출용 team-knowledge-showcase Bank의 실제 v2 Recall. 검색어와 4개의 결과 본문이 함께 보입니다. 원본 PNG는 docs/images/에 보존했습니다.

이 화면은 작성자가 현재 제출용 Bank에서 직접 조회한 검색 증거입니다. 결과 중 v1에 관한 역사적 설명도 함께 검색되므로, 적용 시에는 현재 승인된 v2 원본·적용 범위를 다시 확인해야 합니다. 과거 Agent B의 자동 조회 로그는 별도 실행 기록으로 구분합니다.

3. 사용 예시 및 확인 결과

갱신 후 Agent 판단 변화

이전 B의 기록과 코드를 전달받은 새 Codex 세션에서 v2를 확인했습니다.

변경 전후

시점	Agent의 근거와 판단
v1 게시 후	B는 version이 승인 범위 밖이라 두 항목 동시 안내를 팀 규칙으로 확정하지 않고 보류했습니다.
v2 게시 후	새 Codex 세션이 이전 B의 기록·코드를 읽고 Hindsight와 Git의 `active / v2`를 확인했습니다. 문서 출처와 Git 원본은 `f15b484…`로 일치했습니다.
수정한 예제	두 항목이 없으면 `설정 오류: model, version 항목이 없습니다`를 출력하도록 B의 코드를 수정했습니다. 테스트 5개가 통과했습니다.

이 결과가 보여주는 범위

새 세션이 기록을 읽고 현재 승인된 원본에 맞춰 판단과 코드를 바꿀 수 있었습니다. 같은 대화 세션이 계속 열린 상태에서 기억이 자동 갱신된 것은 아닙니다. 실제 서로 다른 사람의 계정이나 서버 권한도 이 실험에서 검증하지 않았습니다.

> **기록.** v1과 v2 각각의 검색·원본 확인·테스트 내역은 `experiments/config-agent-b/`에 있습니다. 기존 코드와 문서를 비교한 단일 사례이므로 일반적인 성능 우위를 판단할 수 없습니다.

3. 사용 예시 및 확인 결과

지식 ID 없는 검색 실험

기존 B 실험은 ID와 경로를 알려준 한계가 있어, 별도 세션에 업무만 주고 한 번 더 확인했습니다.

실험 조건

별도 Codex 세션에 “설정값이 누락됐을 때 구체적인 오류를 제공하고, 팀이 검증한 방법이 있으면 찾아 적용하라”고 요청했습니다. 프롬프트와 환경 안내에 Bank ID·지식 ID·YAML 경로는 넣지 않았습니다. Hindsight API 주소와 Git checkout 위치, 초기 코드에 든 설정 키는 제공했습니다.

단계별 결과

단계	확인 결과
검색 · 성공	Bank 목록을 조회해 `team-knowledge-showcase`를 선택하고 recall로 `COMP-015 v2`를 찾았습니다.
원본 검증 · 미완료	로컬 main 파일은 읽었지만 근거 커밋을 확인하지 못했고, 원격 main을 새로 확인한 기록도 없습니다.
적용 · 부분 성공	누락 항목을 안내하는 코드를 작성하고 자체 테스트 5개가 통과했습니다. 다만 출력 끝의 마침표가 승인 원본 예시와 달랐습니다.

결과 해석. 관련 지식을 찾고 일부 적용했지만 출처 검증과 정확한 출력 일치는 완성하지 못했습니다. 테스트 통과를 원본과 완전히 일치한다는 증거로 삼지 않습니다.

단일 실행이며 API·Git 위치와 관련 초기 코드를 제공했습니다. 일반적인 자율성이나 검색 우위로 확대하지 않습니다. 실제 요청, 환경 지침, 코드·테스트·trace는 `experiments/blind-discovery/`에 있습니다.

4. 구현 범위와 한계

구현 및 검증 범위

기능의 존재, 실제 실행 결과, 운영 수준의 검증을 구분합니다.

항목	상태
지식 파일·검증기	구현. 스키마와 근거 필드 검사 및 선택적인 원격 커밋 확인.
Skill과 PR	스킬 절차와 draft PR을 사용. 사용자가 PR #5·#6을 직접 병합.
Hindsight 게시·검색	로컬 1.1.0에서 수동 게시와 v1·v2 검색을 확인. 최신 제출용 team-knowledge-showcase Bank에서도 수동 게시·조회.
다른 Agent와 갱신	별도 Codex 세션의 v1 활용과, 이전 기록을 읽은 새 세션의 v2 적용 확인. 같은 열린 세션의 자동 기억 갱신은 미검증.
자동 게시	미구현. 현재 GitHub Actions는 검사만 수행.
사용자·Bank 권한	서버 측 ACL·게시 전용 인증·독립 리뷰 강제 미구현 또는 미검증.
추가 발견 실험	ID 없이 검색해 관련 지식을 찾았으나, Agent는 근거 커밋을 확인하지 못했고 출력 문자열도 원본과 달랐습니다. 단일 실행입니다.
효과 비교	개인 기억·Git·Hindsight의 반복 비교와 시간·비용·정확도 측정은 수행하지 않았습니다.
재현 조건	코드 테스트와 공개 근거 확인은 가능. 실제 게시는 Git checkout과 Hindsight·LLM 환경이 추가로 필요.

로컬 Bank 구분은 보안 격리가 아니며 Publisher의 로컬 잠금은 여러 컴퓨터 사이의 동시 실행을 제어하지 못합니다.

4. 구현 범위와 한계

PR 병합을 API 게시까지 자동화한다면

현재의 수동 Publisher를 확장하는 설계입니다. 자동 게시 워크플로·외부 인증은 아직 구현하지 않았습니다.

이벤트부터 실제 반영까지

PR 제출
CI 형식·테스트 검증
→
사람 Merge
보호된 main 확정
→
main Push
게시 Workflow 시작

Snapshot 검증
SHA·YAML·소유권
→
Hindsight API
추가·교체·폐기
→
결과 확인
로그·재시도·알림

단계	구현 시 동작과 검증 기준
① 제안·검사	`pull_request`의 `validate.yml`에서 Schema·테스트 실행. 쓰기 Secret 없이 검증하고, 통과·사람 검토 전에는 Team Bank를 변경하지 않음.
② 승인된 이벤트	PR 병합으로 `main`이 변경되면 `push: branches: [main]`이 게시 트리거. 직접 Push를 막는 보호 규칙과 필수 검사·리뷰를 별도 설정해야 '승인된 변경만' 보장됨.
③ 동일 원본 검증	게시 실행은 승인된 `main` Commit SHA 하나를 고정. 최신 원격 HEAD·깨끗한 Checkout·전체 YAML 검증 및 `source_repository` 소유권 확인.
④ 변경 계산	`reconcile.py`가 전체 Snapshot과 Bank 관리 문서를 비교. `dry-run`으로 추가·교체·폐기 목록 확인. 다른 저장소가 소유한 문서는 변경하지 않음.
⑤ API 게시	Active는 `POST /v1/default/banks/{id}/memories`, 동일 `document_id`에 `update_mode=replace`. Retired는 `DELETE /v1/default/banks/{id}/documents/{doc_id}`.
⑥ 확인·복구	성공 응답, 문서 ID·Commit·검색 결과 확인 후 완료 기록. 오류 시 실패를 남기고 최신 Snapshot으로 재실행; 부분 성공을 전체 완료로 기록하지 않음.

중요한 구분. GitHub PR의 *Approve*만으로는 게시하지 않습니다. **Merge 후 main 변경**이 게시 이벤트입니다. Hindsight 문서 교체가 이미 열린 Agent 세션이나 개인 Bank의 오래된 기억까지 갱신하는 것은 아닙니다.

4. 구현 범위와 한계

자동 게시를 실제 구축하기 위한 조건

개인 노트북의 로컬 실험과 팀 운영 환경을 분리하고, 인증·실행기·장애 대응을 순서대로 구현합니다.

운영 방식	설정과 현실적인 제약
단계 A · 현재	사람이 PR을 병합하고 로컬에서 `reconcile.py --snapshot-commit`을 실행. Hindsight는 `127.0.0.1:8888`. GitHub에는 게시용 인증정보를 보관하지 않음.
단계 B · 단기 자동화	같은 신뢰된 Mac에서 `launchd` 등의 주기 실행으로 `git fetch`→최신 `main` 확인→전체 Snapshot Publisher를 자동 호출. 네트워크 노출 없이 로컬 API 사용 가능하나 Mac이 꺼져 있으면 지연됨. **미구현.**
단계 C · 운영 자동화	GitHub Actions가 보호된 `main` 변경을 감지해 인증 가능한 Hindsight HTTPS 엔드포인트에 게시. 현재 Client에는 인증 헤더가 없어 먼저 추가·테스트해야 함. Runner의 네트워크 접근과 권한 분리가 전제. **미구현.**

운영 자동화 구현 체크리스트

1. GitHub. `.github/workflows/publish.yml` 추가: `push(main)` + 필요시 `workflow_dispatch`, `contents: read`, `concurrency` 직렬화. `main` PR·검사·리뷰 보호를 설정하고 워크플로 파일도 검토 대상에 포함.

2. 네트워크·인증. GitHub-hosted Runner는 Mac의 `localhost`에 연결할 수 없음. 인증된 HTTPS API 또는 격리된 신뢰 실행기 확보 후, Publisher HTTP Client에 인증 헤더·타임아웃·Secret 마스킹 구현. 공개 저장소의 Self-hosted Runner에 외부 PR 코드를 실행시키지 않음.

3. 게시 안정성. 구버전 작업이 최신 Snapshot을 덮어쓰지 않게 실행 순서 제어·HEAD 재확인, 동일 ID 재시도, 429/5xx 재시도, 문서 폐기 범위 제한. 401/403은 즉시 실패하고 경고.

완료로 인정할 자동화 테스트

(a) 미병합 PR은 Team Bank 무변경 → (b) 병합 후 동일 지식 ID 1건 게시 → (c) 다음 Merge에서 같은 ID 교체 → (d) `retired` 병합 시 소유 문서만 삭제 → (e) 서버 장애·중복 Merge에서 최신 Snapshot으로 복구. 결과에 PR·Commit·Actions Run·API 응답을 함께 기록.

> 자세한 구성 예시와 보안 가정은 `docs/AUTOMATION_DESIGN.md`, 평가자용 Fork·PR·수동 게시 실습은 `docs/GITHUB_SETUP.md`에 정리했습니다.

4. 구현 범위와 한계

개선 우선순위

실제 팀에 도입하기 전 재현성, 권한, 운영 안정성부터 보완합니다.

순서	개선과 검증 기준
1. 재현성	평가자가 Hindsight와 LLM 환경을 준비한 상태에서 설치부터 게시·검색까지 독립 재현하고 기록합니다. 공개 저장소의 코드·근거와 제출 파일을 일치시킵니다.
2. 접근 권한	사용자별 Bank 읽기·쓰기와 게시 전용 자격증명을 적용하고, 권한 없는 직접 접근이 거부되는지 검증합니다.
3. 자동 게시	보호된 main Merge를 트리거로 Snapshot 검증→Hindsight Retain/Replace/Delete→결과 확인·재시도를 구축합니다. 17~18쪽의 상세 API·권한·실행기 계획을 기준으로 검증합니다.
4. 실제 팀 평가	서로 다른 팀원과 다양한 지식으로 리뷰·오래된 지식·검색 실패를 관찰합니다. 기존 열린 Agent 세션에서 정보가 어떻게 갱신되는지도 확인합니다.
5. 대안 비교	같은 업무를 개인 기억, Git 직접 조회, Hindsight+Git 조건에서 반복합니다. 업무 결과뿐 아니라 출처 확인·시간·비용을 측정합니다.

이번 프로토타입의 판단

지식을 저장하는 기능 자체보다, **근거와 적용 범위를 함께 공유하고 새 근거로 판단을 갱신하는 경로**를 작은 예제로 확인했습니다. 운영 환경에 그대로 배포할 수 있는 완성형 서비스로 보지는 않습니다.

5. AI Coding Agent 활용 내역

AI 도구와 활용 내역

Codex를 설계·구현·실험·문서화에 사용하고, 사용자는 방향을 정하고 PR을 병합했습니다.

도구와 역할

Codex 앱은 설계 검토, 코드·테스트 작성, 실행 확인, 보고서와 PR 준비에 사용했습니다. 별도 Codex CLI 세션은 `gpt-5.6-luna`로 B 역할과 v2 후속 검증을 수행했습니다. Hindsight 1.1.0은 로컬 지식 저장·검색에 사용했습니다.

설정 사례의 첫 `$share-knowledge` CLI 실행은 사용량 제한으로 중단됐습니다. 이후 Codex 앱 세션이 생성된 파일을 확인하고 스킬 절차에 따라 YAML과 PR #5를 완성했습니다. 실험 당시 Skill과 제출 저장소용으로 경로를 수정한 사본은 구분합니다.

주요 실제 요청

- 첫 사례: “기존 구현은 … '설정 오류'만 출력하지만 개선된 구현은 '설정 오류: model 항목이 없습니다'라고 구체적으로 알린다.” 실제 입력·출력·테스트와 근거 커밋을 요구.
- B 실행: 팀 Bank recall과 문서 ID·source_snapshot, 최신 Git 원본과 적용 범위를 확인한 뒤 독립 예제와 테스트를 작성하도록 요청.
- v2 후속 실행: “새 Codex 세션이므로 이전 대화가 자동으로 보존됐다고 주장하지 말고 그 기록을 읽어라.” 이전 코드와 지식을 비교하고 수정·검증하도록 요청.

프롬프트와 기록

실제 요청은 `prompts/`, Agent의 확인 과정은 `experiments/config-agent-b/`에 있습니다. 첫 요청에서 이전 사례를 언급한 두 문장만 제출용 발췌에서 생략했고, 각 파일에 구분했습니다. ID를 주지 않은 추가 실험의 요청과 환경 지침도 `experiments/blind-discovery/`에 있습니다. 전체 대화 원문을 제출한 것은 아닙니다.

사람은 사례와 방향을 선택하고 PR을 직접 병합했습니다. 독립 제3자 리뷰·여러 사용자 계정은 확인하지 않았습니다.